```
import itertools

# ========================================================================
# Implementation of Truth-Table Enumeration for Propositional Entailment
# Standard AI Algorithm Framework (Ref: Lab Slide Page 12)
# ========================================================================

def pl_true(sentence, model):
    """
    Evaluates whether a propositional logic sentence holds true within a given model.
    A model is a dictionary mapping symbol strings to Boolean values (True/False).
    """
    if isinstance(sentence, str):
        return model[sentence]

    operator = sentence[0]

    if operator == 'NOT':
        return not pl_true(sentence[1], model)
    elif operator == 'AND':
        return pl_true(sentence[1], model) and pl_true(sentence[2], model)
    elif operator == 'OR':
        return pl_true(sentence[1], model) or pl_true(sentence[2], model)
    elif operator == 'IMPLIES':
        return not pl_true(sentence[1], model) or pl_true(sentence[2], model)
    elif operator == 'BICONDITIONAL':
        return pl_true(sentence[1], model) == pl_true(sentence[2], model)

    raise ValueError(f"Unknown logical connector encountered: {operator}")


def extract_symbols(sentence):
    """Helper to collect all unique atomic symbol names from a formula sentence."""
    if isinstance(sentence, str):
        return {sentence}
    syms = set()
    for part in sentence[1:]:
        syms.update(extract_symbols(part))
    return syms


def print_unified_truth_table(kb, queries, intermediate_sentences):
    """
    Generates and prints ONE comprehensive truth table showing the symbols,
    sub-sentences, KB, and all query evaluation streams side-by-side.
    """
    # Gather all unique symbols across KB and all queries
    all_sentences = [kb] + list(queries.values()) + list(intermediate_sentences.values())
    symbols = set()
    for s in all_sentences:
        symbols.update(extract_symbols(s))
    symbols = sorted(list(symbols))

    # Establish ordered headers for the single matrix view
    headers = symbols + list(intermediate_sentences.keys()) + ["KB"] + list(queries.keys())
    col_widths = [max(len(h), 7) for h in headers]

    # Render table boundaries
    header_row = " | ".join(f"{h.center(w)}" for h, w in zip(headers, col_widths))
    print("\n" + "=" * len(header_row))
    print("📊 UNIFIED TRUTH TABLE MATRIX")
    print("=" * len(header_row))
    print(header_row)
    print("-" * len(header_row))

    # Iterate through all permutations from True to False values
    for combination in itertools.product([True, False], repeat=len(symbols)):
        model = dict(zip(symbols, combination))
        row_values = []

        # 1. Base Symbols
        for sym in symbols:
            row_values.append("true" if model[sym] else "false")

        # 2. Intermediate Sentences
```

```
        for expr in intermediate_sentences.values():
            row_values.append("true" if pl_true(expr, model) else "false")

        # 3. Main KB Evaluation
        kb_val = pl_true(kb, model)
        row_values.append("true" if kb_val else "false")

        # 4. Challenge Queries
        for q_expr in queries.values():
            row_values.append("true" if pl_true(q_expr, model) else "false")

        row_str = " | ".join(f"{v.center(w)}" for v, w in zip(row_values, col_widths))

        # Highlight row channels where KB is a valid model
        if kb_val:
            print(f"{row_str}  <-- [CRITICAL KB MODEL]")
        else:
            print(row_str)

    print("=" * len(header_row) + "\n")


def tt_entails(kb, alpha):
    """Core recursive check to verify if KB entails alpha."""
    def tt_check_all(kb, alpha, symbols, model):
        if not symbols:
            if pl_true(kb, model):
                return pl_true(alpha, model)
            return True
        p = symbols[0]
        rest = symbols[1:]

        model_true = dict(model)
        model_true[p] = True

        model_false = dict(model)
        model_false[p] = False

        return (tt_check_all(kb, alpha, rest, model_true) and
                tt_check_all(kb, alpha, rest, model_false))

    symbols = sorted(list(extract_symbols(kb).union(extract_symbols(alpha))))
    return tt_check_all(kb, alpha, symbols, {})


# ======================================================================
# LAB EXERCISE DRIVER CONFIGURATION
# ======================================================================
if __name__ == "__main__":
    # Sentences from Lab Sheet Page 11
    s1 = ['IMPLIES', 'Q', 'P']
    s2 = ['IMPLIES', 'P', ['NOT', 'Q']]
    s3 = ['OR', 'Q', 'R']

    # Combined Knowledge Base: (Q → P) ∧ (P → ¬Q) ∧ (Q ∨ R)
    kb = ['AND', s1, ['AND', s2, s3]]

    kb_components = {
        "Q → P": s1,
        "P → ¬Q": s2,
        "Q ∨ R": s3
    }

    # Evaluated queries (ii, iii, iv) from Page 11
    queries = {
        "Q_ii: R": 'R',
        "Q_iii: R→P": ['IMPLIES', 'R', 'P'],
        "Q_iv: Q→R": ['IMPLIES', 'Q', 'R']
    }

    # 1. Output the single unified verification grid
    print_unified_truth_table(kb, queries, kb_components)

    # 2. Print final standalone analytical conclusions
    print("📋 FINAL ENTAILMENT INFERENCE RESULTS:")
    print("-" * 50)
    for label, alpha_expr in queries.items():
        result = tt_entails(kb, alpha_expr)
```

```
        status = "✅ YES (KB ⊨ α)" if result else "🔴 NO (Does NOT entail)"
        print(f"• Challenge {label.ljust(12)}: {status}")
    print("-" * 50)
```

```
================================================================================================
📊 UNIFIED TRUTH TABLE MATRIX
================================================================================================
   P    |    Q    |    R    | Q → P  | P → ¬Q | Q ∨ R  |    KB   | Q_ii: R | Q_iii: R→P | Q_iv: Q→R
------------------------------------------------------------------------------------------------
  true  |   true  |   true  |   true |  false |   true |  false  |   true  |    true    |    true
  true  |   true  |  false  |   true |  false |   true |  false  |  false  |    true    |   false
  true  |  false  |   true  |   true |   true |   true |   true  |   true  |    true    |    true    <-- [CRITICAL KB MODEL]
  true  |  false  |  false  |   true |   true |  false |  false  |  false  |    true    |    true
 false  |   true  |   true  |  false |   true |   true |  false  |   true  |   false    |    true
 false  |   true  |  false  |  false |   true |   true |  false  |  false  |    true    |   false
 false  |  false  |   true  |   true |   true |   true |   true  |   true  |   false    |    true    <-- [CRITICAL KB MODEL]
 false  |  false  |  false  |   true |   true |  false |  false  |  false  |    true    |    true
================================================================================================

📋 FINAL ENTAILMENT INFERENCE RESULTS:
--------------------------------------------------
• Challenge Q_ii: R     : ✅ YES (KB ⊨ α)
• Challenge Q_iii: R→P  : 🔴 NO (Does NOT entail)
• Challenge Q_iv: Q→R   : ✅ YES (KB ⊨ α)
--------------------------------------------------
```